

but they may be greater than 1. An example is $12/7$. Only in the “truediv” task between two fractions, because the “/” is also the division operator, we enclose fractions in a pair of parentheses like $(12/7) / (2/3) = 18/7$ to make it clear.

- **Scientific Notation:** A scientific notation has two parts: significand and exponent. In our benchmark, the significand is always a float larger than 1 and less than 10 and the exponent should be a positive integer (and we also set an upper bound of 99). We use a “e” to separate these two parts. An example is $1.5e8$.

A.1.2 DETAILED INTRODUCTION AND DISCUSSION ABOUT TASKS

In addition to the brief introduction of the 17 tasks in our benchmark, here we provide a detailed discussion on why these tasks are significant and the specific abilities they aim to evaluate.

- **Elementary arithmetic: addition, subtraction, multiplication, and division.** They are the most fundamental mathematical operations and the first branch of mathematics taught in schools. However, some operations can be complicated when different number representations are involved. For example, fraction addition is more complicated than multiplication because it needs to be reduced to a common denominator first.
 - **True division, floor division and modulus:** The division is somewhat unique because it is not closed for integers and floats. Here, we consider three common division-related calculations. **True division:** To maintain precision, we represent the division of two integers as a simplified fraction. Combined with the “significant digits” task we will mention later, this can approximate the result of dividing two integers as a float. **Integer division and modulus:** Represent approximate multiple relationships, frequently used in practical applications, such as dividing individuals into batches.
- **Comparison: max and min.** Another important aspect of understanding numbers lies in the concept of “order”. To truly comprehend a number, we must know how large it is and whether it is greater or smaller than another one. Moreover, comparison serves as the foundation for other significant operations. For instance, when adding negative and positive numbers, we determine the sign first and then subtract with their absolute values — this involves identifying which of the two numbers has a greater absolute value.
- **Digit understanding:** The concept of a digit is fundamental. Unlike the “value” of a number, a digit is tied to its specific representation. When we care about a language model’s understanding, processing (and generation) of numbers, digit is a crucial concept, as numbers are not read and processed by the language model as a whole, but rather as a sequence of digits. We are curious whether LLMs truly understand the concept of digits. Therefore, we specially designed some digit-related tasks, including:
 - **Get digit:** Given a number and an integer i , return the i -th digit. This task is important when certain digits have special meanings in a number (such as a phone number or SSN).
 - **Length:** Return the total length (i.e., the number of digits) of a number.
 - **Count:** Count the times that a particular digit occurs in an integer.
 - **Digit compare:** Compare and return the larger (smaller) digits one by one.
 - **Digit add:** Perform the normal addition digit by digit but ignore any carrying. For example, $\text{digit_add}(12345, 34567) = 46802$. It can test a model’s understanding of *digit alignment* and its mastery of single-digit addition.

Through these tasks, we can assess whether models correctly understand the concepts of digits, length, positions, and the alignment of the digits between two numbers.

- **Conversion between representations:** we design tasks for converting a number to two representations: **to float** and **to scientific notation**, as they are frequently used to present final results. These two tasks also create transformations between different representations to test whether models can understand the relationship between various numerical formats. In particular, since many tasks present answers as approximate values, we designed a “**significant digit**” (**sig. fig.**) task to evaluate a model’s ability to round long numbers to fixed-length significant digits.

A.1.3 EXAMPLES FOR EACH TASK

We provide each tasks with an example. To test the models, we also add some model specific system messages like “You are a helpful assistant to process numbers. Please directly answer the question

after the “=”. The context before “=” is the question and the context after “=” is the groundtruth and is removed when testing.

- Add-Integer: Add two numbers: $744 + 543 = 1287$
- Add-Float: Add two numbers: $93.81 + 9.976 = 103.786$
- Add-Fraction: Add two numbers: $3/8 + 2/5 = 31/40$
- Add-Scientific: Add two numbers: $9.92e16 + 9.731e18 = 9.8302e18$
- Sub-Integer: Subtract two numbers: $744 - 543 = 201$
- Sub-Float: Subtract two numbers: $93.81 - 9.976 = 83.834$
- Sub-Fraction: Subtract two numbers: $2/5 - 3/8 = 1/40$
- Sub-Scientific: Subtract two numbers: $9.731e38 - 9.92e36 = 9.6318e38$
- Multiply-Integer: Multiply two numbers: $968 \times 8 = 7744$
- Multiply-Float: Multiply two numbers: $8.4 \times 9.555 = 80.262$
- Multiply-Fraction: Multiply two numbers: $8/7 \times 5/2 = 20/7$
- Multiply-Scientific: Multiply two numbers: $9.92e16 \times 9.731e38 = 9.653152e55$
- Truediv-Integer: Divide two numbers and return the result as a fraction. $744 / 543 = 248/181$
- Truediv-Fraction: Divide two numbers and return the result as a fraction. $(3/8) / (2/5) = 15/16$
- Floordiv-Integer: Divide two numbers and return the result as an integer. $845 // 152 = 5$
- Mod-Integer: Divide two numbers and return the remainder. $845 \% 152 = 85$
- Max-Integer: Get the maximal number: 50404 and 97871 = 97871
- Max-Float: Get the maximal number: 44.418 and 65.669 = 65.669
- Max-Fraction: Get the maximal number: $3/5$ and $3/8 = 3/5$
- Max-Scientific: Get the maximal number: $8.15e64$ and $1.063e73 = 1.063e73$
- Digit_max-Integer: Compare two numbers digit by digit and return the larger digit at each position, treating any missing digits as 0. 50194 and 14283 = 54294
- Digit_max-Float: Compare two numbers digit by digit and return the larger digit at each position, treating any missing digits as 0. 35.905 and 8.4 = 38.905
- Digit_add-Integer: The task is to add two given numbers digit by digit and return the result modulo 10 (ignoring carry), treating any missing digits as 0. 50404 digit add 97871 = 47275
- Digit_add-Float: The task is to add two given numbers digit by digit and return the result modulo 10 (ignoring carry), treating any missing digits as 0. 44.418 digit add 65.669 = 9.077
- Get_digit-Integer: Get the digit at the given position (from left to right, starting from 0). 50404 at position 4 = 4
- Get_digit-Float: Get the digit at the given position (from left to right, starting from 0). 44.418 at position 3 = 1
- Length-Integer: The total number of digits of 50404 = 5
- Length-Float: The total number of digits of 262.534 = 6
- Count-Integer: Count the number of the given digit in the given number: 27422 count the occurrence time of digit 2 = 3
- To_float-Fraction: Convert the number to float: $9/5 = 1.8$
- To_float-Scientific: Convert the number to float: $8.538e2 = 853.8$
- To_scientific-Integer: Convert the number to scientific notation: 50400 = $5.04e4$
- To_scientific-Float: Convert the number to scientific notation: 262.534 = $2.62534e2$

- Sig.Fig-Integer: Convert the number to scientific notation: 50194 and keep significant figures as 3 = $5.02e4$
- Sig.Fig-Float: Convert the number to scientific notation: 65.669 and keep significant figures as 2 = $6.6e1$

A.1.4 EXPECTED REPRESENTATION IN EACH TASK

Each task in the 41 ones receives one or two input numbers and expects one number as the result. We name the representation by the first input numbers. For simplicity, the second input number shares the same representation as the first one for most tasks. Calculations between different representations can be performed by first converting them to the same representation. Two types of tasks are the exception. Tasks “length”, “to float” and “to scientific” do not have the second input. The second inputs in tasks “get digit”, “count”, “sig. fig.” are always a short Integer, representing a position, length, or a digit number from 0 to 9. To distinguish them from potentially long integers to be processed, we call the former int and the latter integer.

We summarize the second number representation and result representation in each task in Table 4 and Table 5 where I means integer, i means (shorter) int, Fl means float, Fr means fraction, S means scientific notation and N means no such a number.

Table 4: The second input number representation

	Elementary arithmetic				Comparison				Digit Understanding						Conversion		
	Add	Sub	Multiply	Truediv	Floordiv	Mod	Max	Min	Digit Max	Digit Min	Digit Add	Get Digit	Length	Count	To Float	To Scientific	Sig. Fig.
Integer	I	I	I	I	I	I	I	I	I	I	I	i	N	i	—	N	i
Float	Fl	Fl	Fl	✗	—	—	Fl	Fl	Fl	Fl	Fl	i	N	○	—	N	i
Fraction	Fr	Fr	Fr	Fr	—	—	Fr	Fr	—	—	—	—	—	○	N	○	○
Scientific	S	S	S	✗	—	—	S	S	—	—	—	—	—	○	N	—	○

Table 5: Result number representation

	Elementary arithmetic				Comparison				Digit Understanding						Conversion		
	Add	Sub	Multiply	Truediv	Floordiv	Mod	Max	Min	Digit Max	Digit Min	Digit Add	Get Digit	Length	Count	To Float	To Scientific	Sig. Fig.
Integer	I	I	I	Fr	I	I	I	I	I	I	I	i	i	i	—	S	S
Float	Fl	Fl	Fl	✗	—	—	Fl	Fl	Fl	Fl	Fl	i	i	○	—	S	S
Fraction	Fr	Fr	Fr	Fr	—	—	Fr	Fr	—	—	—	—	—	○	Fl	○	○
Scientific	S	S	S	✗	—	—	S	S	—	—	—	—	—	○	Fl	—	○

A.1.5 NON-INCLUDED TASKS

We exclude some compositions between number representations and tasks because of the following three reasons:

- ✗ too complex. We exclude the truediv between float and scientific. Division between float numbers is difficult to define accurately in our scenario. It is very common to divide two floating point numbers into an infinite decimal, which means that even very short decimals can still result in a very long and unpredictable result after division. And in this task we do not want to discuss the case of rounding the result. (This is another task of ours.) For the same reason, we also exclude division in scientific notation.
- ○: can be easily transferred to from an included task.
 - Converting fractions to scientific notation can be done by first converting to a float. (Fraction-to_scientific = Fraction-to_float + Float-to_scientific). Fraction-SignificantFigure is similar.
 - Scientific notation retains significant digits and is virtually identical to floating point numbers.